

EMU-ZEBU搭建配置

1 概述

EMU平台仿真，是最接近真实硬件条件的仿真方法。算子级别的EMU仿真，不需要关注太多EMU平台细节，重点是在各方搭建、验证完毕的组件基础上，按照流程配置环境，并执行算子的编译与分析。

本文仅针对Zebu的相关流程进行介绍。

2 权限申请

首先需要申请Zebu资源的权限，申请链接如下：

<https://siorigin.feishu.cn/share/base/form/shrcn5YXs9HabXsCU7kgOlj2qGe>

平台类型选择“zebu”，申请提交后，和HAV团队沟通，等待审批通过（一般由IT反馈）

3 环境登入

使用Zebu资源，需要通过ETX方式登入，选择符合条件的终端设备（一般为RD05~RD07），激活后，通过terminal查看权限情况（如提示无权限，需要和HAV团队沟通权限申请进展情况）：

代码块

```
1  ls /hav_farm/release
2  bsub -q zeburun -Is ls
```

4 环境配置

登入环境后，验证权限无问题，可开始进行环境配置，配置前需要选择HAV版本，具体可参考HAV的版本发布文档：[SIPU v1.5 release](#)

本文档采用的Zebu版本如下：

代码块

```
1  # 0930版本
```

```
2 DB_PATH=/hav_farm/release/sipu/sipu_v1p5_95nl_0930/zebu/s1g1c4e_vhost_fw2
```



1. 注意，以下操作需要在个人目录中执行，即`cd ~`
2. 环境配置分为“首次配置”与“已有目录运行”，请根据实际情况进行操作
3. 本地创建的目录名称，可根据需要进行修改，此处使用的名称为`run_folder`

(1) 首次配置

代码块

```
1 # 镜像拷贝
2 cp /hav_farm/release/lib/zebu/ubuntu22.04.qcow2 ~/
3 # 创建目录
4 mkdir run_folder
5 # 进入目录，并确保在csh下
6 cd run_folder
7 csh
8 # 初始化运行目录，运行初始化脚本
9 $DB_PATH/run_init.csh
10 # 获取环境变量
11 source zs5_v0.csh
```

(2) 已有目录运行

代码块

```
1 # 进入目录，并确保在csh下
2 cd run_folder
3 csh
4 # 获取环境变量
5 source zs5_v0.csh
```

5 安装固件

在当前目录下，参考执行以下安装指令：

代码块

```
1 cd ~/run_folder
2 csh
3
4 git clone https://gitlab.siorigin.com/sw/fw/firmware_release.git
5
6 # X_Y_Z 为当前release的版本
7 # <plat>: pldm_zebu
8 source firmware_release/mscp/SIPU_EMU_A0_X_Y_Z/setup.csh <plat>
```

固件一般只需要安装1次，不需要更新升级的情况下，不需要重新安装。

6 设备选择

配置完成后，可以开始进行Zebu设备的检查与选择，首先查看Zebu空闲情况：

代码块

```
1 check_hw
```

检查结果中，U*.M*.S*表示该Zebu资源已占用，zeburun0*表示该server已占用。

使用空闲资源：

代码块

```
1 # 假定U1.M0.S0设备空闲
2 set_hw U1.M0.S0
```

选择空闲server，运行zebu：

代码块

```
1 # 假定server 1空闲
2 zs5_run1 make run
```

7 启动vHOST

PCIE枚举完成，看到terminal打印“Waiting for client connection on port xxx”后，执行如下步骤：

(1) 新开一个terminal，并进到相同目录

代码块

```
1 cd ~/run_folder
2 csh
3 source zs5_v0.csh
```

(2) 使用与运行Zebu相同的runtime server

代码块

```
1 # 当前terminal与host建立连接, 只需ping一次
2 zs5_run* make ping_host
3 # 启动vHost (账号: ceshi, 密码: ceshi)
4 zs5_run* make run_host
```

启动成功后，会弹出QEMU（Ubuntu系统），后续操作需要在此环境中进行。

(3) 文件共享

由于QEMU中，不便于与服务器之间进行数据拷贝，可使用共享目录的方式进行文件互访，在QEMU系统界面上，启动terminal，然后执行以下指令（folder_name按自己需要命名）：

代码块

```
1 sudo mkdir -p /mnt/<folder_name>
2 sudo mount -t 9p -o trans=virtio,version9p2000.L SharedFolder
  /mnt/<folder_name>
```

执行完成后，可通过以下目录与QEMU进行文件共享：

代码块

```
1 ~/run_folder/ShareFolder <-----> /mnt/<folder_name>
```

8 安装驱动



以下内容仅供参考，最新内容请参阅文档：[SIPU 驱动与运行时 发布说明](#)

首先安装相关依赖：

代码块

```
1 # 修改apt 指向本地源
2 vim /etc/apt/sources.list
3 deb [arch=amd64] http://10.192.3.67/ubuntu jammy main restricted universe
  multiverse
4 deb [arch=amd64] http://10.192.3.67/ubuntu jammy-updates main restricted
  universe multiverse
5 deb [arch=amd64] http://10.192.3.67/ubuntu jammy-security main restricted
  universe multiverse
6 deb [arch=amd64] http://10.192.3.67/ubuntu jammy-proposed main restricted
  universe multiverse
7 deb [arch=amd64] http://10.192.3.67/ubuntu jammy-backports main restricted
  universe multiverse
8 deb [arch=amd64 trusted=yes] http://10.192.3.67/sio stable main
9 sudo apt update
10
11 # KMD依赖包安装
12 sudo apt update
13 sudo apt install libcunit1-dev curl sparse autoconf dkms linux-headers-$(uname
  -r) linux-modules-extra-$(uname -r) cmake
14 # sirt依赖包安装 (如需源码编译)
15 sudo apt install libgtest-dev
16 sudo apt install cmake cmake-data libjsoncpp25 librhash0 libiberty-dev pkg-
  config libelf-dev
17 # runtime依赖包安装 (如需源码编译)
18 sudo apt install ninja-build python3-jinja2
19 sudo apt install -y gcc-13 g++-13
```

按照以下指令安装驱动并加载（QEMU vHOST环境执行）：

代码块

```
1 # 下载驱动包
2 # 0930版本
3 wget -c https://artifactory.siorigin.com/sio/candidate/driver/sipu-
  dkms_1.0.0_amd64.deb
4
5 # 安装依赖
6 sudo apt install -f
7
8 # 安装驱动(第一次安装需要参考下面Option选项，安装驱动依赖库)
9 # 0930版本
10 sudo dpkg -i sipu-dkms_1.0.0_amd64.deb
11
```

```
12 # 加载驱动
13 modprobe -v sipu
```

9 安装SDK



以下内容仅供参考，最新内容请参阅文档：[📄 SIPU 驱动与运行时 发布说明](#)

按以下指令执行：

代码块

```
1 # 0930版本
2 wget -c
  https://artifactory.siorigin.com/sio/candidate/sdk/sipu_sdk_emulation_latest
3 mkdir /path/to/sipu_sdk_emulation
4 tar -xzvf sipu_sdk_emulation_latest -C /path/to/sipu_sdk_emulation
5 soruce /path/to/sipu_sdk_emulation/sipu_sdk_setup.sh
```

10 验证与跑测



以下内容仅供参考，最新内容请参阅文档：[📄 SIPU 驱动与运行时 发布说明](#)

按以下指令执行：

代码块

```
1 # 检查驱动状态
2 dmesg | grep successfully
3 [ 47.559182] sipu 0000:11:00.0: sipu: driver initialized successfully
4
5 # 查看设备信息
6 ls -la /dev/sipu*
7 total 0
8 drwxr-xr-x  2 root root      80  9月 19 12:01 .
9 drwxr-xr-x 19 root root   4060  9月 19 12:01 ..
10 crw-rw-rw-  1 root root 241,  0  9月 19 12:01 usipu0
11 crw-rw-rw-  1 root root 241, 256  9月 19 12:01 usipu-ctl
```

```
12
13 # 验证 SMI 工具
14 # 工具支持功能和使用说明文档见本文档 SMI工具 章节
15 sipu-smi
```

11 sikernel case测试状态

12 波形分析

- 波形抓取

首先，需要启动EMU终端，vHOST终端，以及QEMU终端，确保各个环节启动正常，ssh能够访问QEMU环境。

然后，在EMU终端中，zRci %提示符后，输入以下指令：

代码块

```
1 # 全波形抓取
2 start_qiwc_dump kernel名称
3 # 部分波形抓取
4 start_fwc_dump kernel名称
```

启动成功后，在QEMU环境中，开始执行需要抓取波形的算子，等待仿真完成，完成后输入以下指令：

代码块

```
1 # 全波形抓取
2 stop_qiwc_dump
3 # 部分波形抓取
4 stop_fwc_dump
```

此时，可以查看工作目录下是否生成了ztdb目录，形如：

代码块

```
1 <your_workspace>/rundir/emulation_data_XXXX/kernel名称.ztdb
```

- 波形转换

生成的波形为ztdb格式，需转成zwd格式后才能用verdi打开，在terminal下输入如下命令（需要source zs5_v0.csh环境变量）：

代码块

```
1 cd <your_workspace>
2 csh
3 source zs5_v0.csh
4 make zSim ztdb=emulation_data_xxx/kernel名称 jobs=10
```

可以通过检查<your_workspace>下的zSimzilla.log查看转波形的进度，关键词“Overall”，"Overall 100"说明波形已转成功（转换时间较长）。

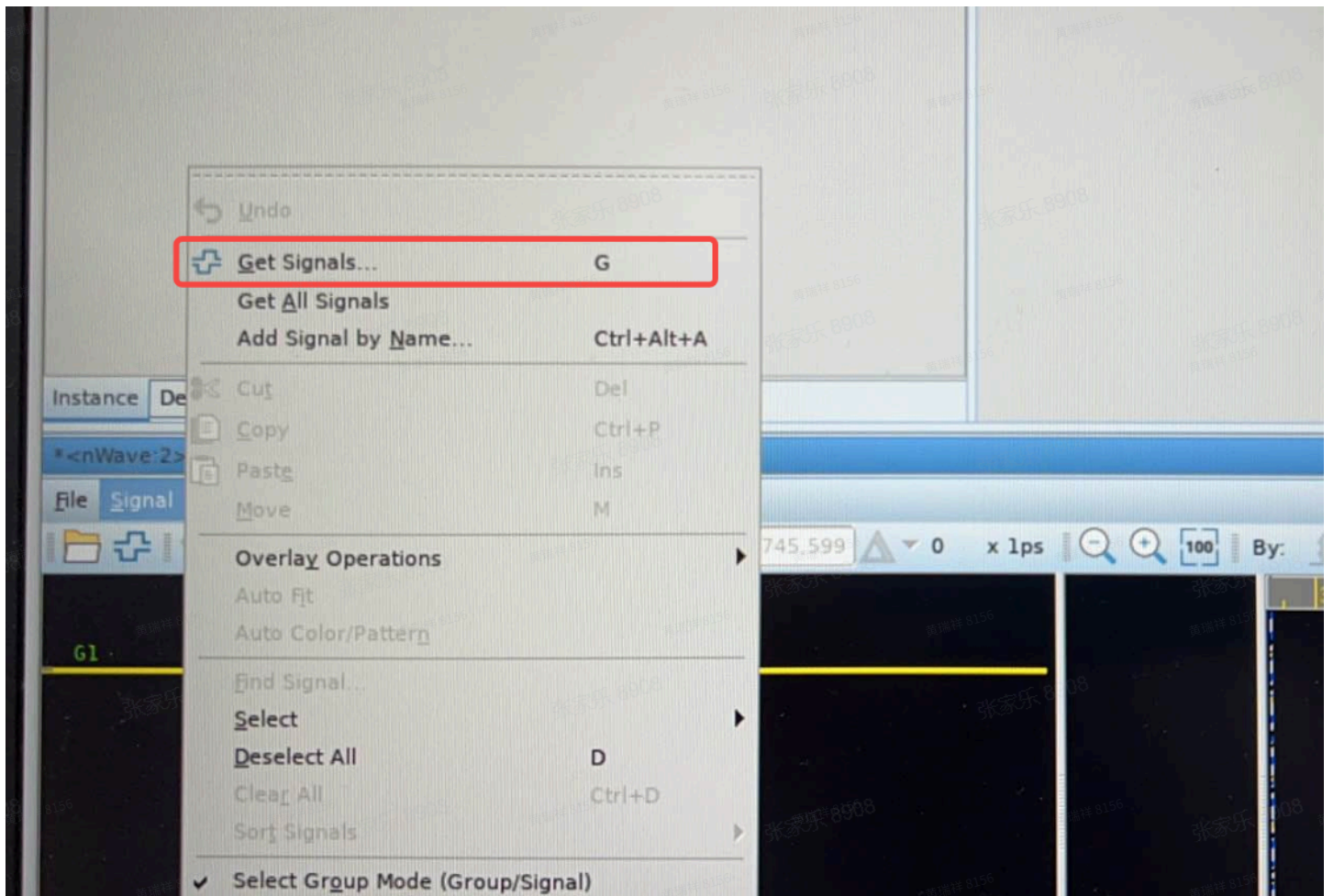
- 波形查看

波形转换成功后，可以使用verdi打开zwd格式波形，命令如下（不要source zs_v0.csh环境变量）：

代码块

```
1 //获取verdi license. 新开terminal
2 csh
3 source /data/tools/modules/init/csh
4 module load verdi/2023.12.sp2
5
6 //打开波形
7 bsub -Is verdi -ssf <zwd_path>.zwd &
```

verdi打开后，需要在波形栏中，选择Signal->Get Signals，在弹出的对话框中，选择关注的信号并添加到波形栏中，来进行波形的观察与分析，如下图所示：



附录

在完成权限申请后，假定空闲资源为zebu (U1.M0.S0)、server (zs5_run1)，简化版执行流程如下：



1. 如果脚本方式运行失败，可尝试手动逐条指令执行，并将前置设定的变量替换为实际字符串
2. 需要在csh模式下执行指令
3. 检查工作目录下的Makefile，ZEBU_PCIE配置是否为1
4. Firmware source完成后，需要手动更新src/env/reset.tcl文件，修复相关问题的文件可从以下位置获取（**0930版本无需此操作**）：
 - a. /vd_farm/zhonglingli/rc_file/reset.tcl
5. QEMU的用户名、密码为：ceshi, ceshi
6. 需要根据实际版本和环境需要，进行变量的设定，下面将列出主要变量
7. DB_PATH与FM_PATH，根据实际需要进行版本选择

主要变量列表：

代码块

```
1  set X_ZEBU_PATH = $DB_PATH
2  set X_FM_REPO = https://gitlab.siorigin.com/sw/fw/firmware_release.git
3  set X_FM_PATH = $FM_PATH
4  set X_FM_PLAT = zebu
5  set X_FM_DB = s1g1c4e_vhost
6  set X_SERVER = zs5_run1
7  set X_ZEBU_DEV = U0.M0.S0
8  set X_RUN_DIR = zwork
9  set X_SERVER_NAME = sio-emu-zeburun01
```

- EMU端

(1) 首次登入

代码块

```
1  cd ~
2  mkdir -p $X_RUN_DIR
3  cd $X_RUN_DIR
4  $X_ZEBU_PATH/run_init.csh
5  source zs5_v0.csh
6  git clone $X_FM_REPO
7  source $X_FM_PATH $X_FM_PLAT $X_FM_DB
8  # 更新buf fix文件，0930版本无需此操作
9  # cp /data/users/zhangjiale/backup/reset.tcl src/env/reset.tcl
10 set_hw $X_ZEBU_DEV
11 eval $X_SERVER make run
```

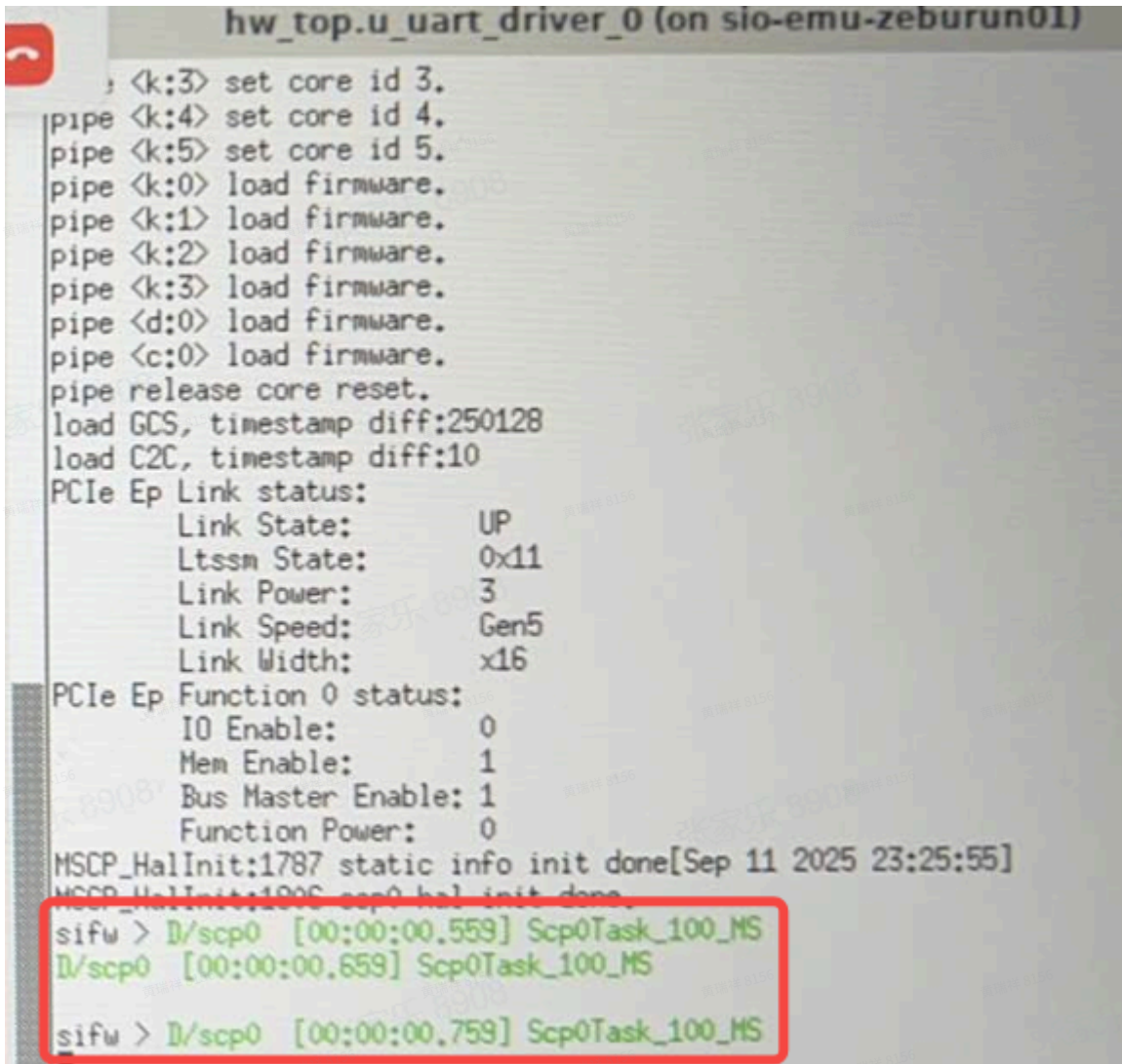
(2) 重复登入

代码块

```
1  cd ~/$X_RUN_DIR
2  source zs5_v0.csh
3  set_hw $X_ZEBU_DEV
4  eval $X_SERVER make run
```

- vHOST端

注意，需要等firmware加载完成后，才可启动vHOST连接，即下图中出现sifw或scp字样以后



(1) 首次登入

代码块

```
1 cd ~/$X_RUN_DIR
2 source zs5_v0.csh
3 eval $X_SERVER make ping_host
4 eval $X_SERVER make run_host
```

(2) 重复登入

代码块

```
1 cd ~/$X_RUN_DIR
2 source zs5_v0.csh
3 eval $X_SERVER make run_host
```

```
4 # ssh -p 3022 ceshi@$X_SERVER_NAME
```

- QEMU端

打开terminal，执行以下命令：

(1) 首次登入

代码块

```
1 # 修改apt 指向本地源
2 vim /etc/apt/sources.list
3 deb [arch=amd64] http://10.192.3.67/ubuntu jammy main restricted universe
  multiverse
4 deb [arch=amd64] http://10.192.3.67/ubuntu jammy-updates main restricted
  universe multiverse
5 deb [arch=amd64] http://10.192.3.67/ubuntu jammy-security main restricted
  universe multiverse
6 deb [arch=amd64] http://10.192.3.67/ubuntu jammy-proposed main restricted
  universe multiverse
7 deb [arch=amd64] http://10.192.3.67/ubuntu jammy-backports main restricted
  universe multiverse
8 deb [arch=amd64 trusted=yes] http://10.192.3.67/sio stable main
9
10 # KMD依赖包安装
11 sudo apt update
12 sudo apt install libcunit1-dev curl sparse autoconf dkms linux-headers-$(uname
  -r) linux-modules-extra-$(uname -r) cmake
13 # sirt依赖包安装 (如需源码编译)
14 sudo apt install libgtest-dev
15 sudo apt install cmake cmake-data libjsoncpp25 librhash0 libiberty-dev pkg-
  config libelf-dev
16 # runtime依赖包安装 (如需源码编译)
17 sudo apt install ninja-build python3-jinja2
18 sudo apt install -y gcc-13 g++-13
19
20 # 下载驱动包，需要根据当前版本选择对应的driver版本，参见第8节
21 # wget -c https://artifactory.siorigin.com/sio/sipu_driver/sipu-
  dkms_1.0.0_amd64_latest.deb
22
23 # 安装依赖
24 sudo apt install -f
25
26 # 安装驱动(第一次安装需要参考下面Option选项，安装驱动依赖库)，需要根据当前版本选择对应的
  driver版本，参见第8节
```

```

27 # sudo dpkg -i sipu-dkms_1.0.0_amd64_latest.deb
28
29 # 加载驱动
30 sudo modprobe -v sipu
31
32 # 安装SDK, 需要根据当前版本选择对应的driver版本, 参见第9节
33 # wget -c
34   https://artifactory.siorigin.com/sio/sipu_sdk_emulation/sipu_sdk_emulation_latest
35   mkdir -p ~/sdk/
36   tar -xzvf sipu_sdk_emulation_latest -C ~/sdk/
37   source ~/sdk/sipu_sdk_setup.sh

```

(2) 重复登入

代码块

```

1 # 加载驱动
2 sudo modprobe -v sipu
3
4 # 配置SDK环境
5 source ~/sdk/sipu_sdk_setup.sh

```

- 简要汇总



1. 可根据需要, 开启第四个或更多的terminal
2. 一旦算子执行失败, 会导致环境异常, 需要重启所有流程, 即关闭各个环境, 并按下面流程再次启动一遍
3. 如需要退出环境, 释放资源, 可点击关闭第二个terminal中弹出的QEMU UBUNTU GUI界面, 然后点击关闭第一个terminal中弹出的UART界面, 然后在第一个terminal中exit退出即可
4. QEMU环境配置连接成功后, 可使用sipu runtime的sample case来验证基础功能, 一般1分钟左右可执行完成: `./build/bin/tests/samples/0_Introduction/vectorAdd/vectorAdd`

当EMU环境配置成功, 且首次vHOST连接成功后, 在EMU环境未变化的情况下, 可按照下面的次序进行环境的启动与算子的编译与验证:

代码块(1) 第一个terminal (不可关闭)

2 `cd ~/zwork` (进入本地的EMU工作目录, 根据实际目录修改)

3 `csh`

4 `source zs5_v0.csh`

5 `set_hw U*.M*.S*` (选择空闲zebu, 星号替换为相应数字)

6 `zs5_run* make run` (选择空闲server, 星号替换为相应数字)

7

8 等待弹出的UART白色对话框中, `sifw >`字样出现, 再开启第二个terminal (约15分钟)

9

10 (2) 第二个terminal (不可关闭)

11 `cd ~/zwork` (进入本地的EMU工作目录, 根据实际目录修改)

12 `csh`

13 `source zs5_v0.csh`

14 `zs5_run* make run_host` (使用与第一个terminal相同的server)

15

16 等待QEMU UBUNTU图形界面出现, 再开启第三个terminal

17

18 (3) 第三个terminal (可关闭, 可重复连接)

19 `ssh -p 3022 ceshi@sio-emu-zeburun0*` (最后的星号数字, 与第一个terminal的server编号一致, 密码为ceshi)

20 `cd ~`

21 `sudo modprobe -v sipu`

22 `source sdk/sipu_sdk_setup.sh` (加载sdk的依赖路径, 根据实际目录修改)

23 `export SIRT_SHIM_NAME=ksipu`

24 然后可开始进行算子代码的编译与运行, 可通过dmesg信息查看是否有错误发生